

CS330 Operating Systems and Lab.

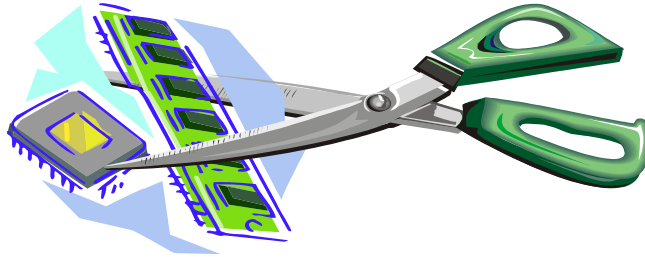
Memory Management

Spring 2026

KAIST

Instructor : Insik Shin

Virtualizing Resources



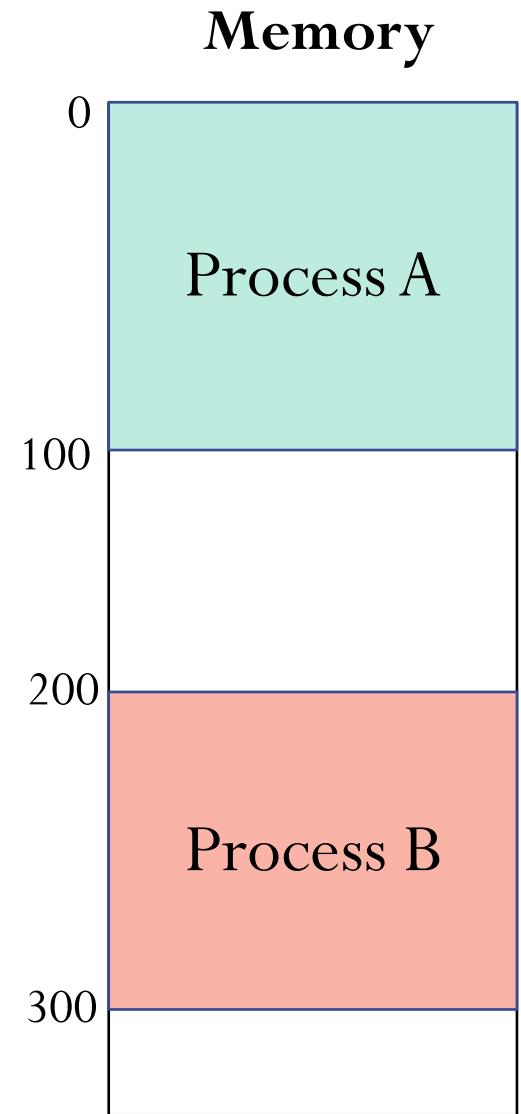
- Physical Reality: Processes/Threads share the same hardware
 - Need to multiplex CPU (CPU Scheduling)
 - Need to multiplex use of Memory (Today)
- Why worry about memory multiplexing?
 - The complete working state of a process and/or kernel is defined by its data in memory (and registers)
 - Consequently, cannot just let different processes use the same memory
 - Probably don't want different processes even to have access to each other's memory (protection)

Important Aspects of Memory Management

- **Sharing with isolation**
 - Multiple processes coexist in main memory
 - Processes should not collide in physical memory
- **Protection**
 - Cannot corrupt OS or other processes
 - Privacy: cannot read data of other processes
- **Transparency**
 - Processes not aware that memory is shared
 - Run regardless of number and/or locations of processes
- **Efficiency**: should have reasonable performance
 - Purpose of sharing is to increase efficiency
 - Do not waste CPU or memory resources

Transparency

- Process can run anywhere in memory (can't predict in advance)
- How ?
 - Relocation
- When to relocate ?
 - Compile time, Load time, or Execution time



Process A

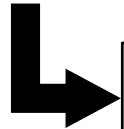
```
int count;
```

```
.
```

```
.
```

```
.
```

```
count = 11;
```



STR 11, @count



STR 11, @100

Memory

0

100

count

Process A

```
int count;
```

```
.
```

```
.
```

```
.
```

```
count = 11;
```

STR 11, @count

STR 11, @100

STR 11, @600

Memory

0

Process B

500

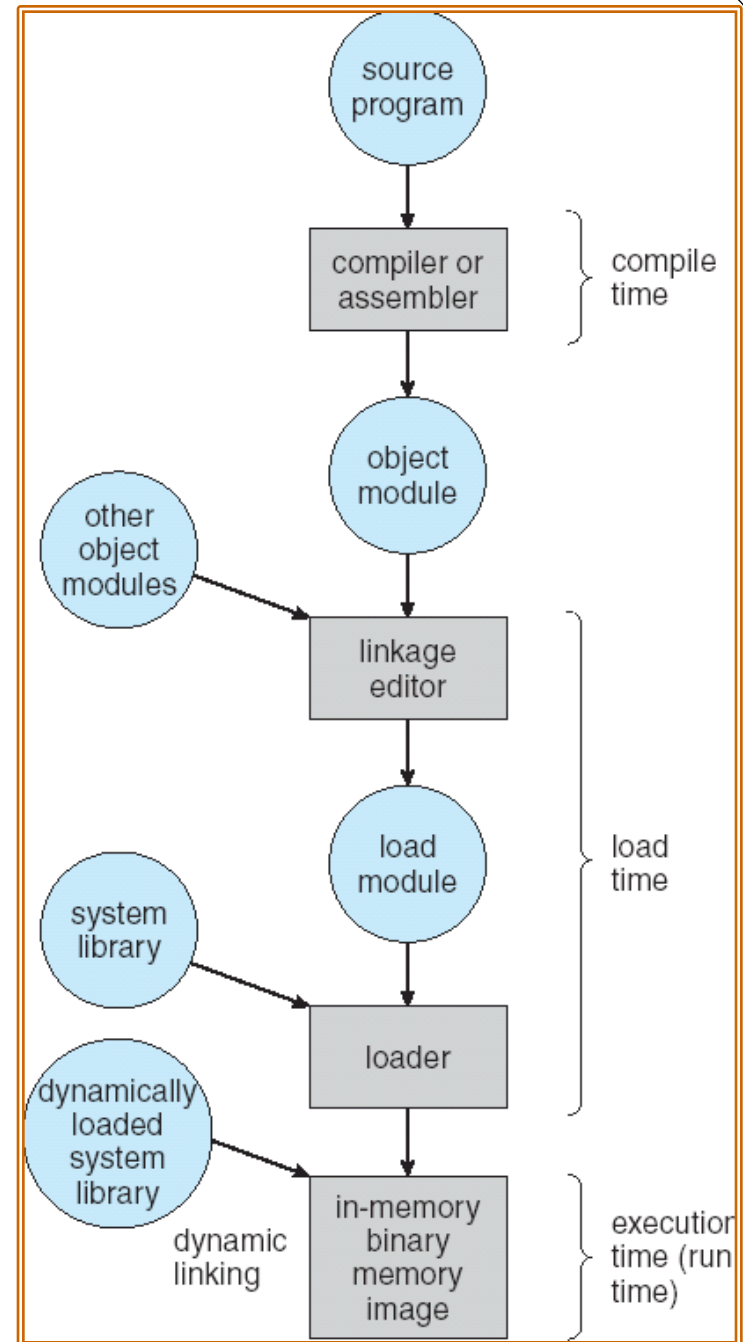
Process A

600

count

Background

- **Compiler** compiles source files into object files
- **Linker** links object files and system libraries into a program
- **Loader** loads program and dynamically-linked library (from disk) into memory and places them in a process for it to be run



Background - Static Linking

- Under *static linking*, system language libraries are
 - treated as any other object module and combined by the linker into the binary program
 - included in each program that uses the lib.

Background - Dynamic Linking

- Linking postponed until execution time
 - typically for system libraries
- Implementation
 - Small piece of code, *stub*, (included in the program binary) used to locate the appropriate memory-resident library routine
 - Stub replaces itself with the address of the routine, and executes the routine
- Advantage
 - useful for libraries: updating the library (such as bug fixing) updates all programs that use it (**shared libraries**)
 - Saves space (multiple processes share one copy of the library in memory)
- Disadvantage
 - Difficult to manage dependencies
 - Runtime failures

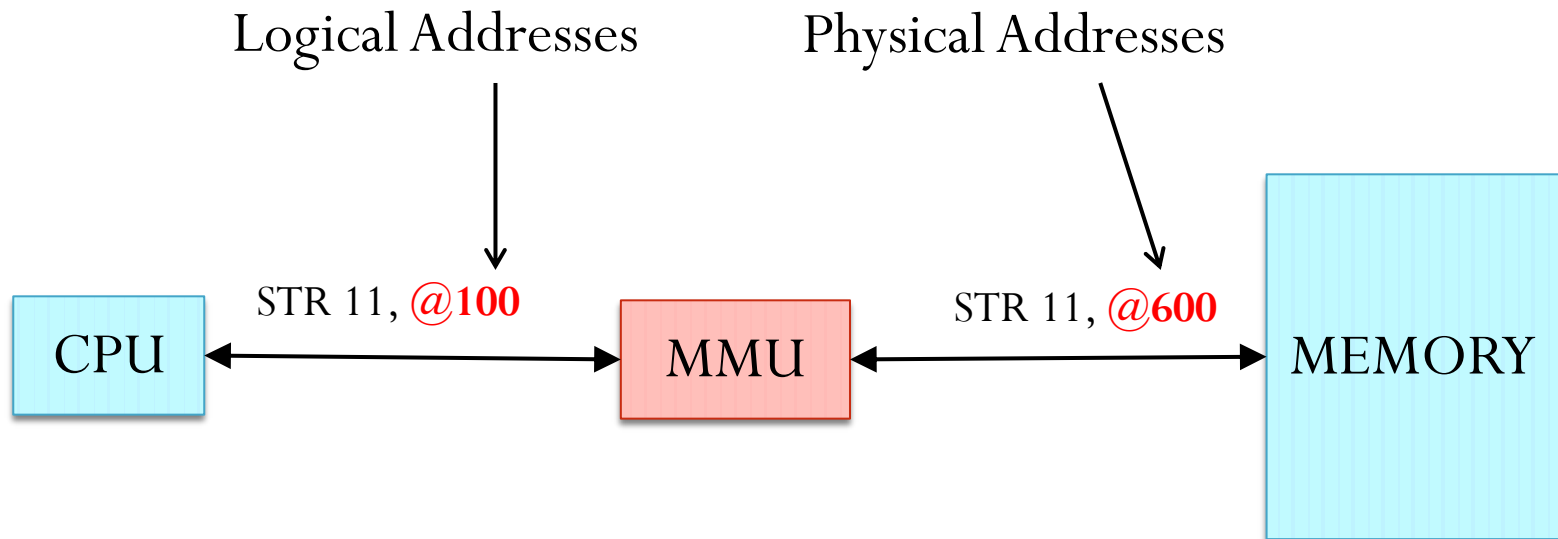
When to Relocate?

- Compile time
 - Hardwire physical location at compile time (**absolute code**)
 - Problem
 - Each program needs to be written with others in mind
 - Not really transparent
- Load time
 - Compiled code relocatable (**relocatable code**)
 - How? All addresses relative to a start address. Change start address to relocate
 - Problem
 - Once loaded, can't change or move
- Execution time
 - Can move during execution
 - This is what's generally done, but need hardware support

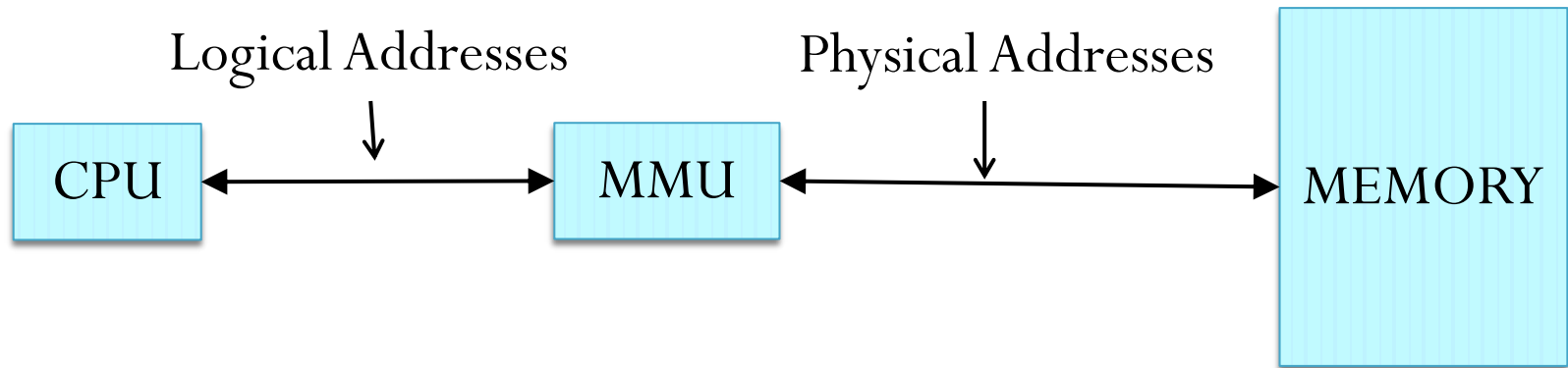
Relocation at Execution Time



Relocation at Execution Time

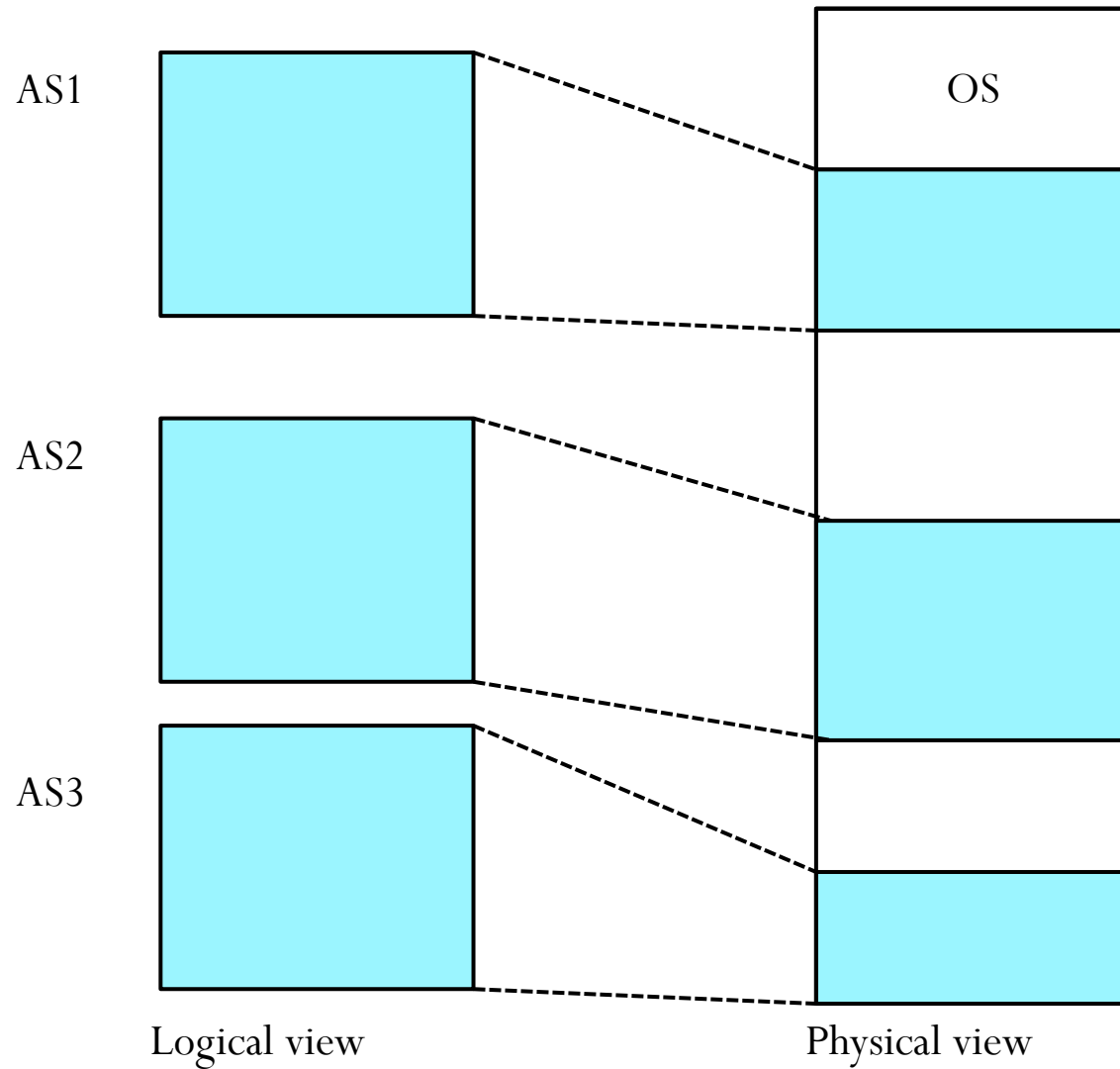


Relocation at Execution Time



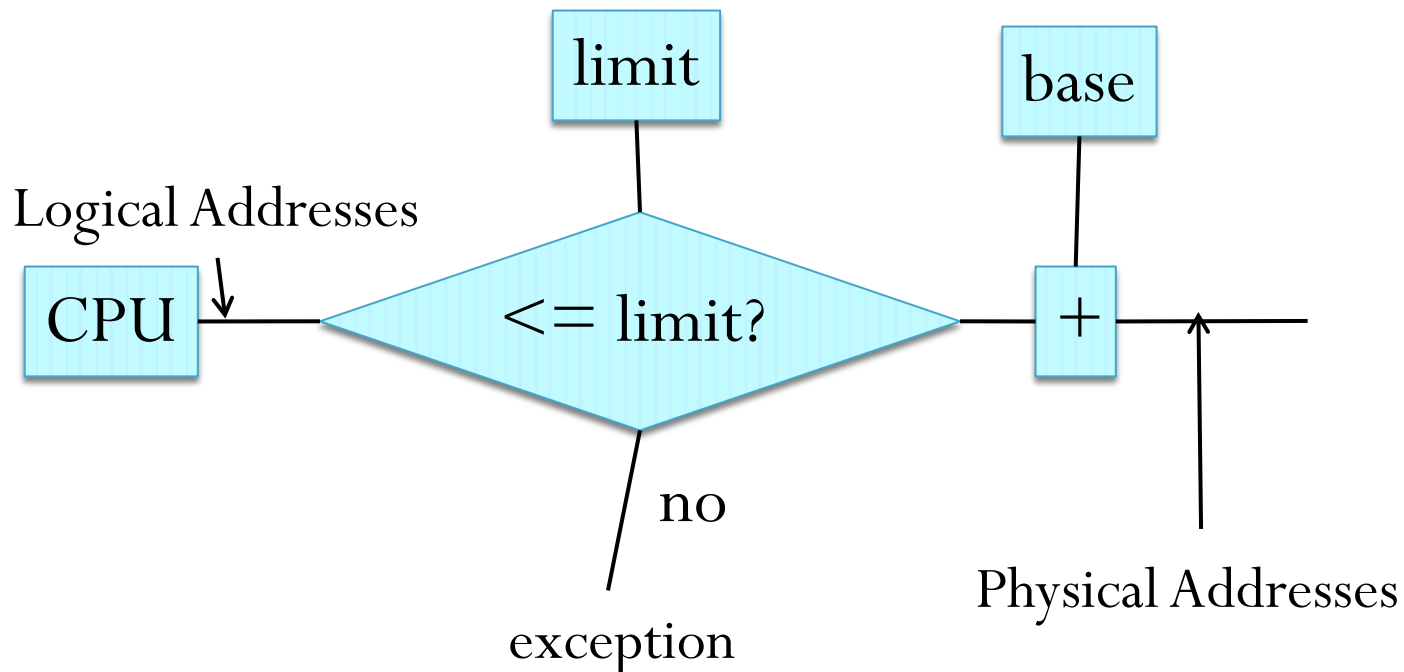
- Map program-generated address to hardware address dynamically at every reference
 - MMU: Memory Management Unit
 - Controlled by OS
- Program: **logical (virtual) address**
- Hardware: **physical (real) addresses**
- **Address space**: each process's view of memory

Address Spaces

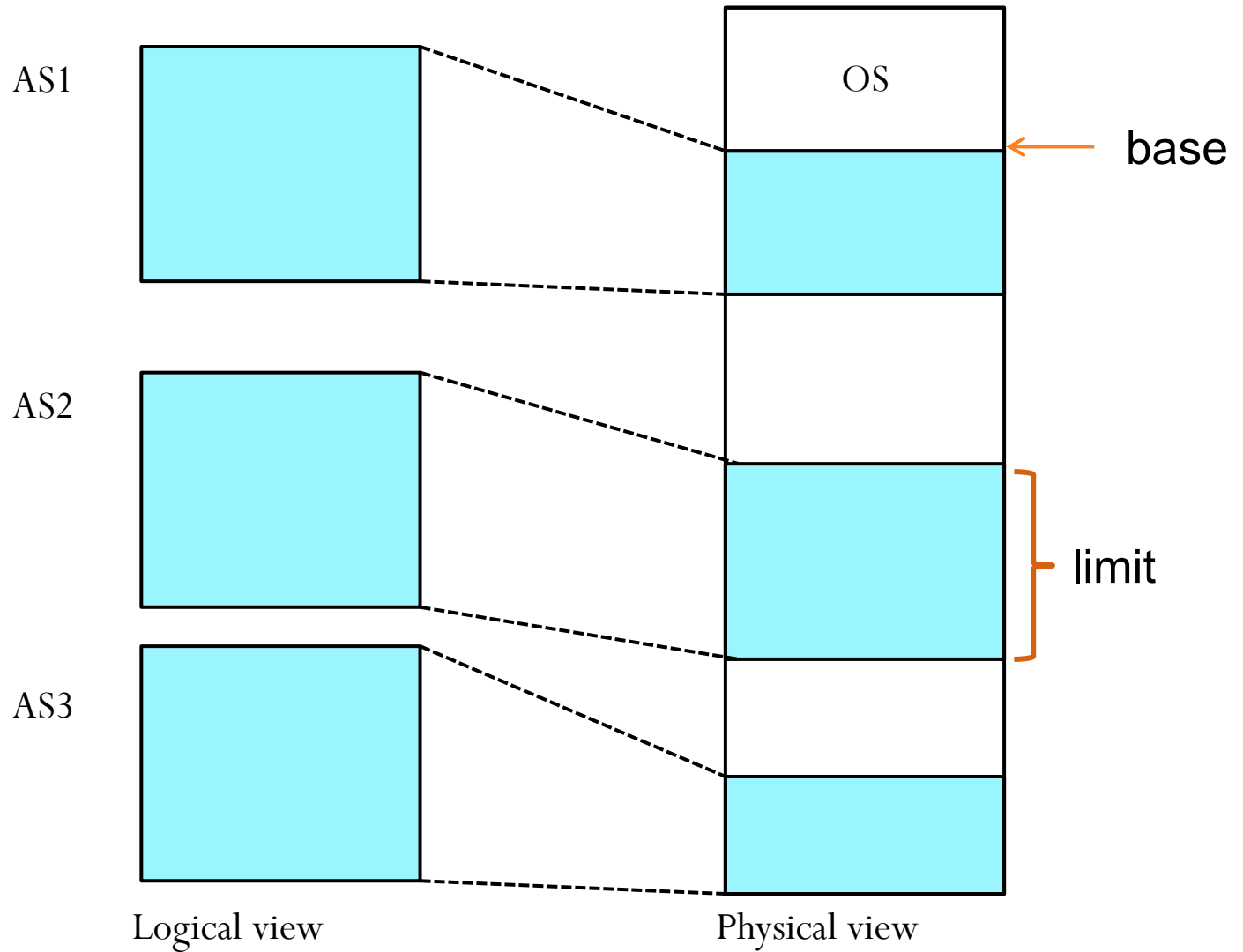


Hardware Support

- How to implement protection?
 - **Base register**: start physical location of address space
 - **Limit register**: last valid address the process may access
 - Appears to have private memory of size equal to limit register

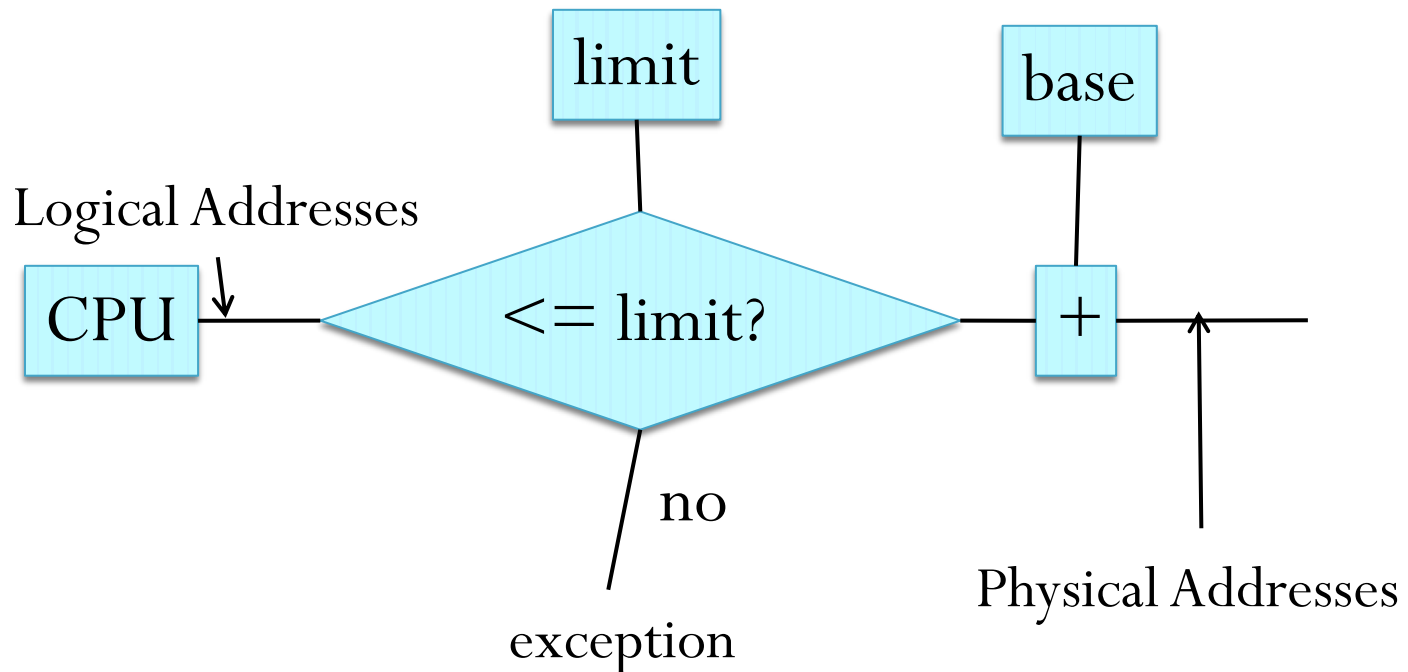


Address Spaces



Implementation

- Translation on every memory access
 - Compare logical address to **limit** register
 - If greater, generate exception
 - Add **base** register to logical addr to generate physical address



Hardware Support

- Two operating modes
 - **User mode**: when user processes run
 - Performs translation of logical address to physical address
 - Protects OS and other processes physical memory
 - **Privileged (protected, kernel) mode**: when OS runs
 - When trap into OS (system calls, interrupts, exceptions)
 - Allows certain instructions to be executed
 - Performs translation of logical address to physical address
 - Allows OS to access all of physical memory

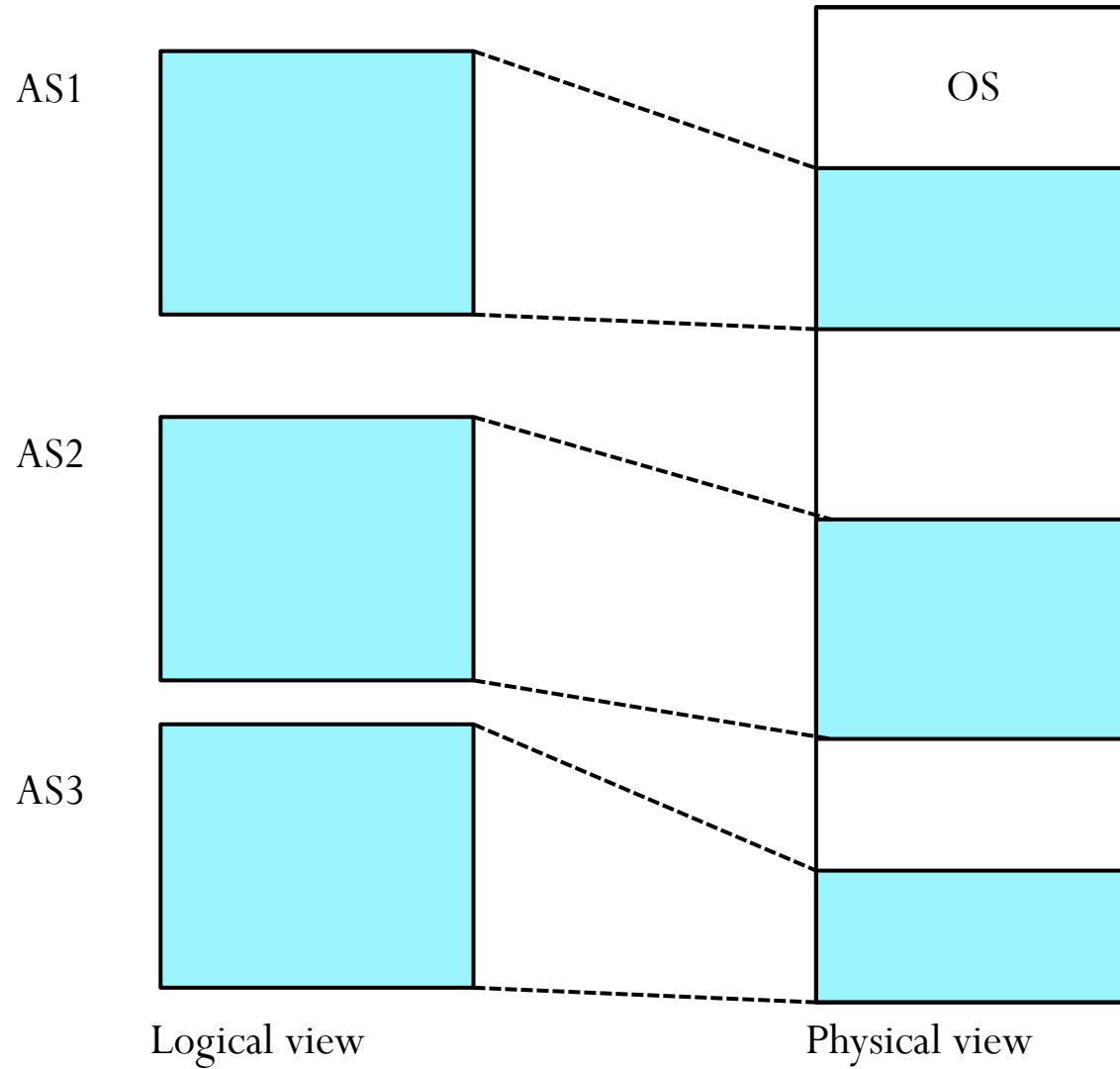
How to Manage Base & Limit Registers?

- Who and when should change base and limit registers ?

Managing Processes with Base and Limit

- Context switch
 - Every context switch, base and limit registers should change
- Protection requirement
 - User process cannot change base and limit registers
 - User process cannot change to privileged mode

Address Spaces



Contiguous Memory Allocation(1)

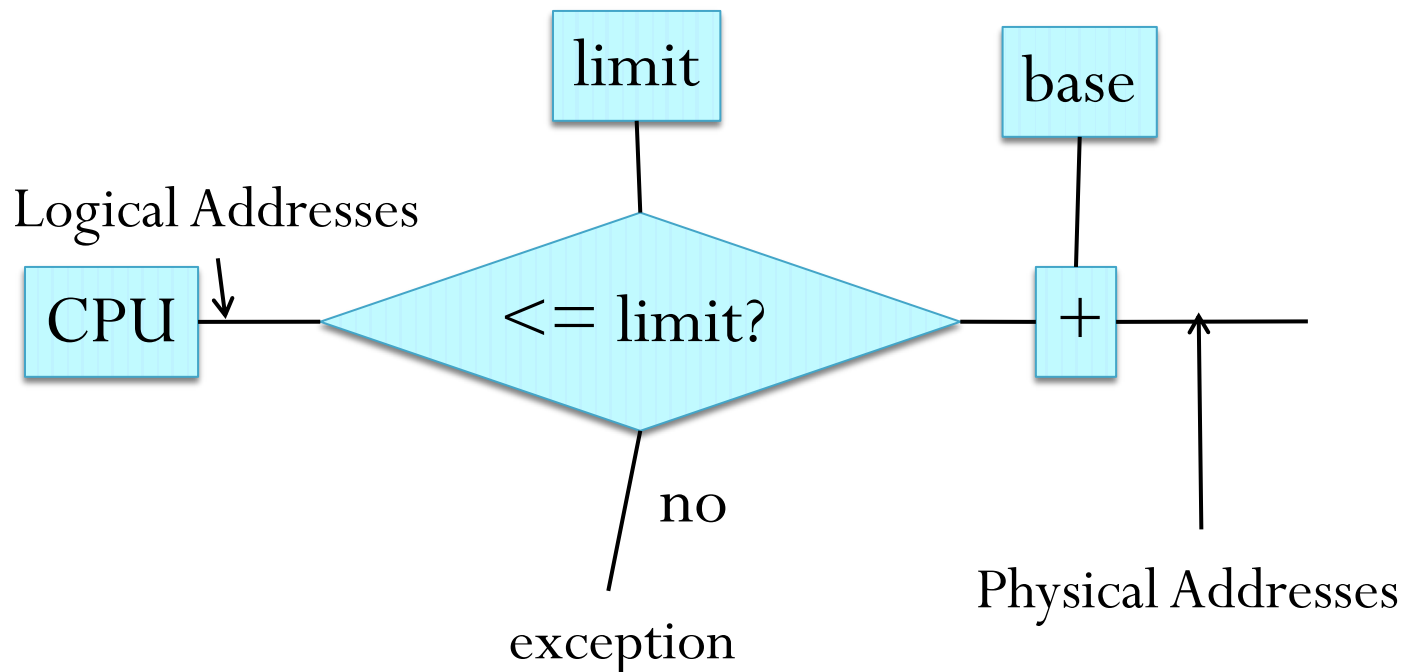
- Background
 - The main memory must accommodate both OS and various user processes. We therefore need to allocate the parts of the main memory in the most efficient way possible
 - In **contiguous memory allocation**, each process is contained in a single contiguous section of memory

Contiguous Memory Allocation (2)

- Main memory usually into two partitions:
 - Resident operating system, usually held in low memory with interrupt vector
 - User processes then held in high memory
- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
 - Base register contains value of smallest physical address
 - Limit register contains range of logical addresses – each logical address must be less than the limit register
 - MMU maps logical address *dynamically*

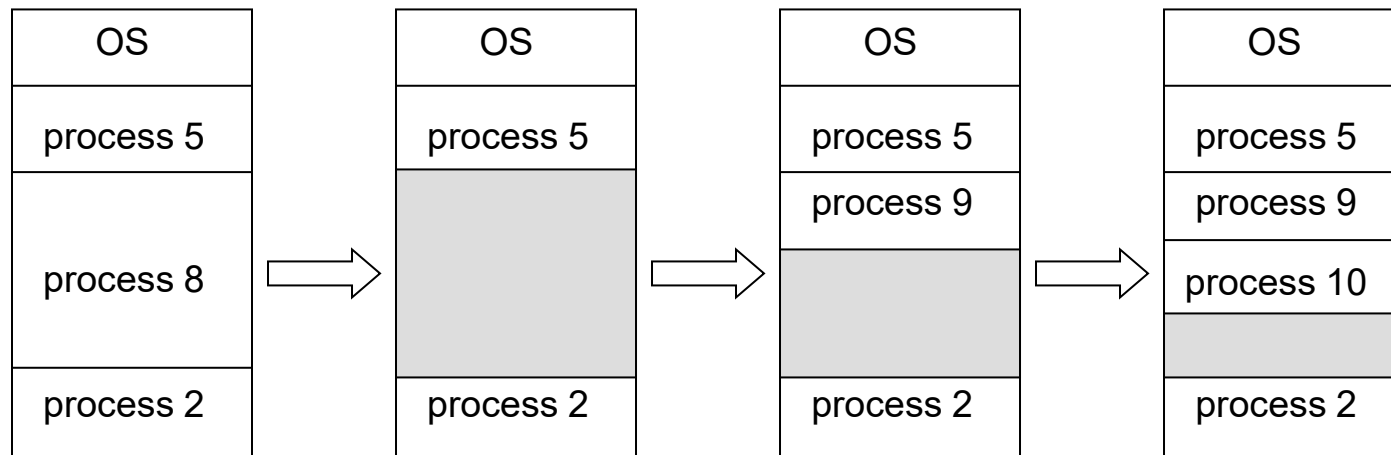
Implementation (revisited)

- Translation on every memory access
 - Compare logical address to **limit** register
 - If greater, generate exception
 - Add **base** register to logical addr to generate physical addr



Contiguous Allocation (Cont)

- Multiple-partition allocation
 - Hole – block of available memory; holes of various size are scattered throughout memory
 - When a process arrives, it is allocated memory from a hole large enough to accommodate it
 - Operating system maintains information about:
 - a) allocated partitions b) free partitions (hole)



Dynamic Storage-Allocation Problem

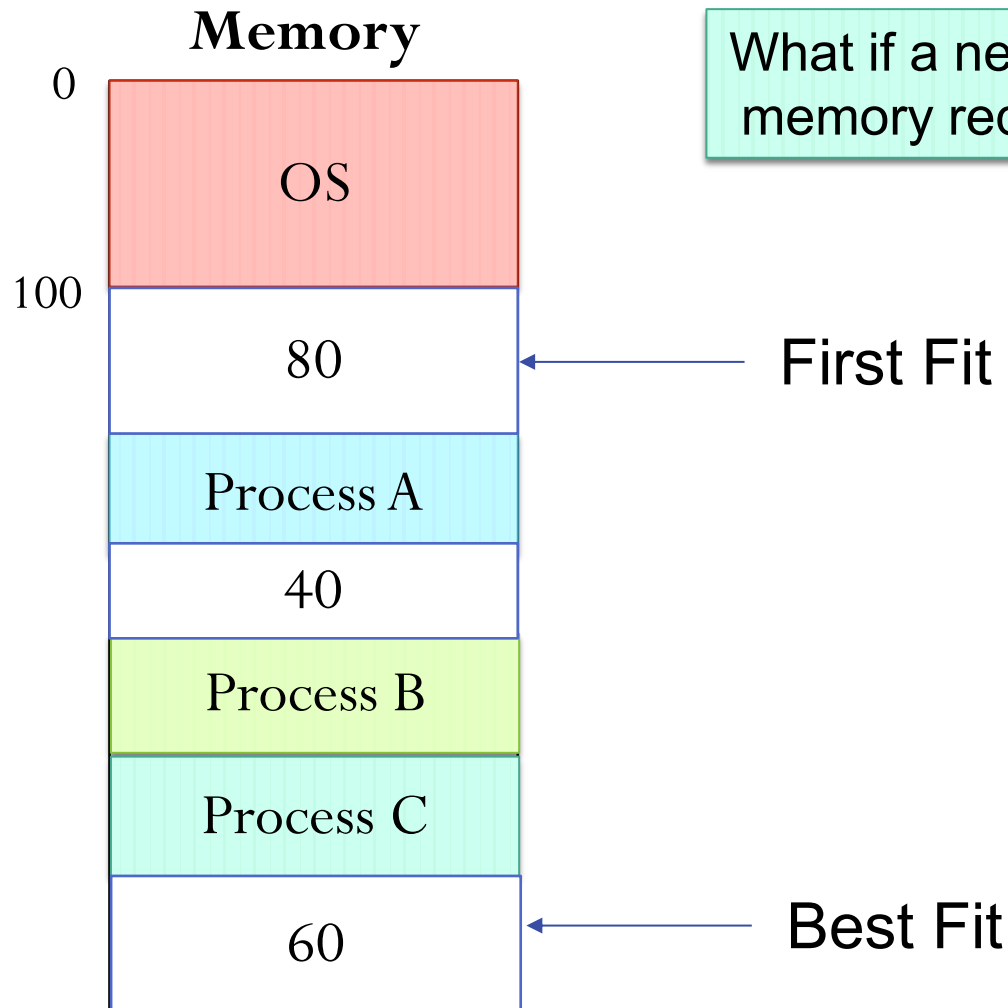
How to satisfy a request of size n from a list of free holes

- **First-fit:**
 - Allocate the *first* hole that is big enough
- **Best-fit:**
 - Allocate the *smallest* hole that is big enough; must search entire list, unless ordered by size
 - Produces the smallest leftover hole
- **Worst-fit:**
 - Allocate the *largest* hole; must also search entire list
 - Produces the largest leftover hole

First-fit and best-fit better than worst-fit in terms of speed and storage utilization

Example

- New process with memory requirement 50



What if a new process with memory requirement 100?

Fragmentation

- **Fragmentation**

- a phenomenon in which (storage) space is used inefficiently, reducing storage capacity and in most cases performance

- **External Fragmentation**

- total memory space exists to satisfy a request, but it is not contiguous

- **Internal Fragmentation**

- allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used

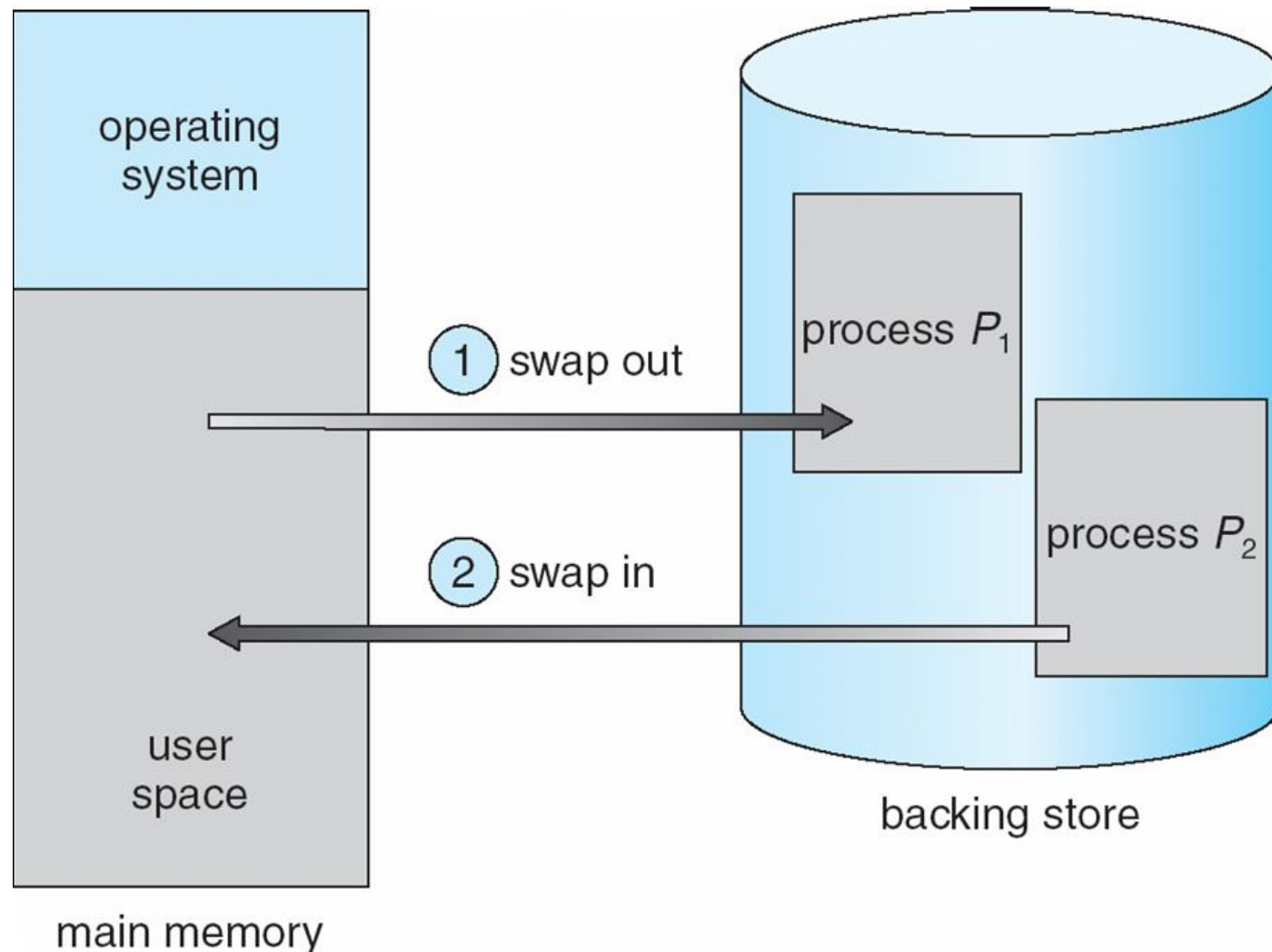
- Reduce external fragmentation by **compaction**

- Shuffle memory contents to place all free memory together in one large block
- Compaction is possible *only* if relocation is dynamic, and is done at execution time

Swapping

- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution
- For example, assume a multiprogramming environment with a round-robin CPU scheduling algorithm. When a quantum expires, the memory manager will start swap out the process that just finished and to swap another process into the memory space that has been freed

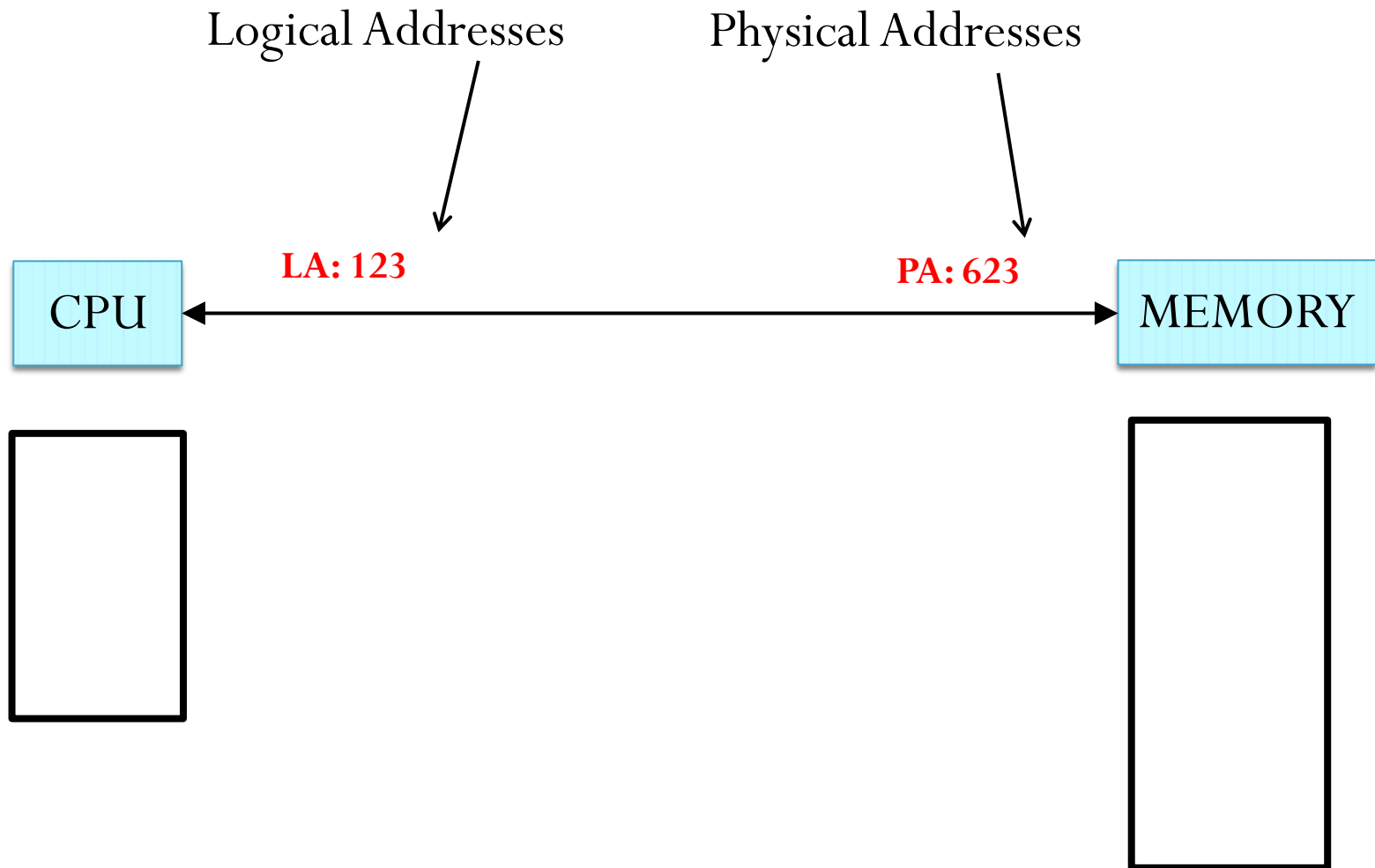
Schematic View of Swapping



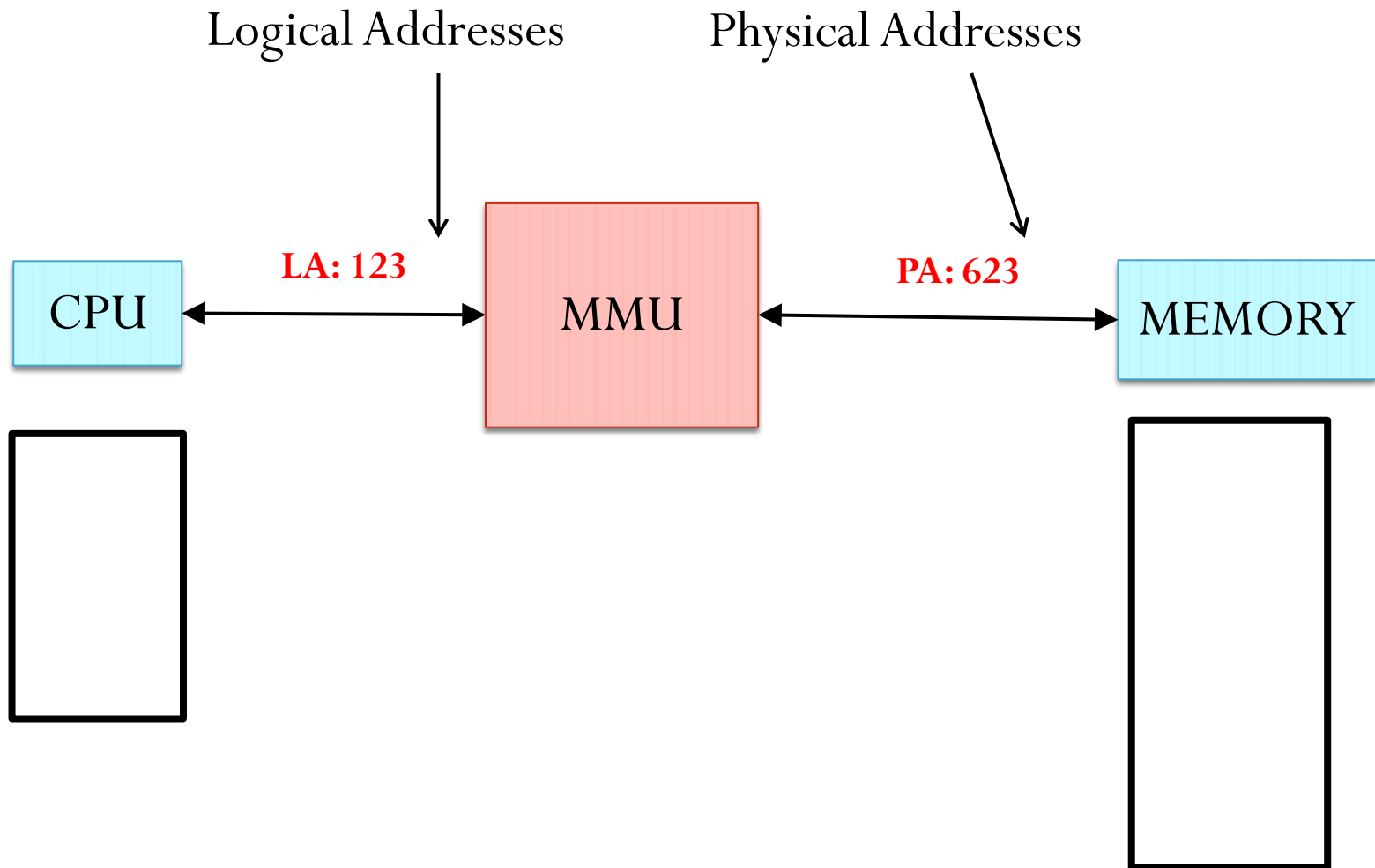
What else to resolve fragmentation?

- Something else to resolve fragmentation?

Relocation at Execution Time



Relocation at Execution Time



Fragmentation

- **Fragmentation**

- a phenomenon in which (storage) space is used inefficiently, reducing storage capacity and in most cases performance

- **External Fragmentation**

- total memory space exists to satisfy a request, but it is not contiguous

- **Internal Fragmentation**

- allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used

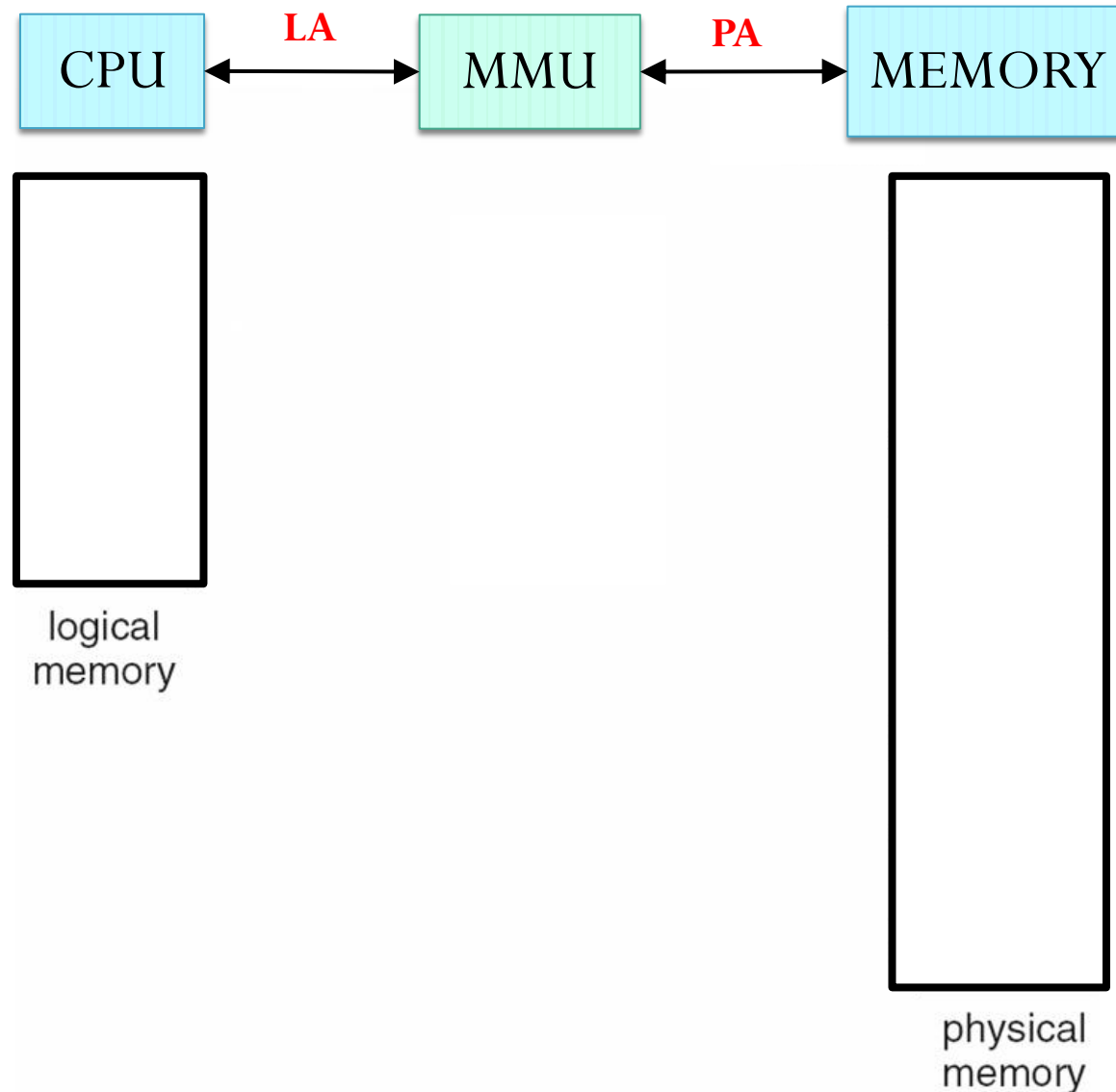
- Reduce external fragmentation by **compaction**

- Shuffle memory contents to place all free memory together in one large block
- Compaction is possible *only* if relocation is dynamic, and is done at execution time

Paging

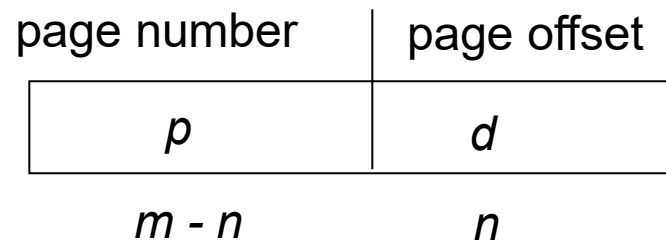
- A memory-management scheme that permits physical address space of a process to be *noncontiguous*
 - Divide physical memory into fixed-sized blocks called **frames** (size is power of 2, between 512 bytes and 8,192 bytes)
 - Divide logical memory into blocks of same size called **pages**
 - Keep track of all free frames
 - To run a program of size **n** pages, need to find n free frames and load program
 - Set up a page table to translate logical to physical addresses
 - Internal fragmentation

Paging Model of Logical and Physical Memory



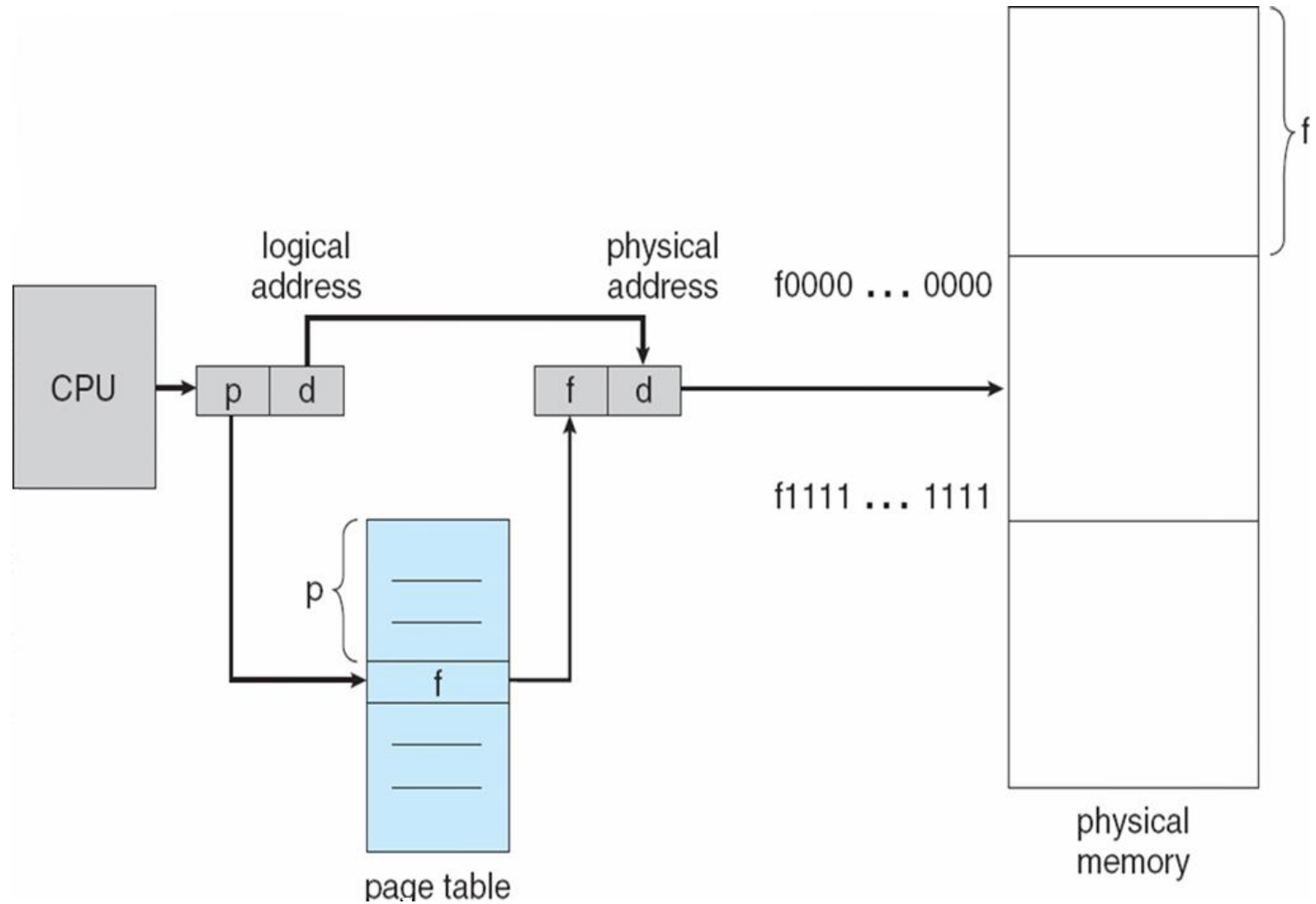
Address Translation Scheme

- Address generated by CPU is divided into:
 - **Page number (p)** – used as an index into a *page table* which contains base address of each page in physical memory
 - **Page offset (d)** – combined with base address to define the physical memory address that is sent to the memory unit



- For given logical address space 2^m and page size 2^n

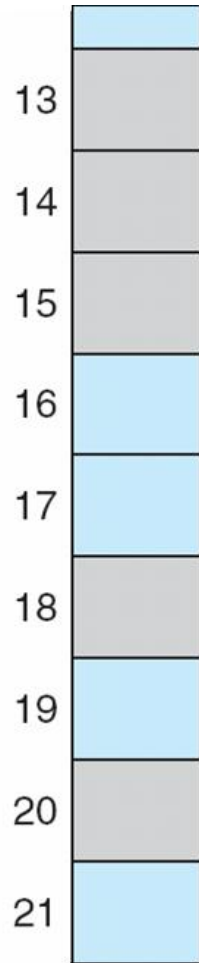
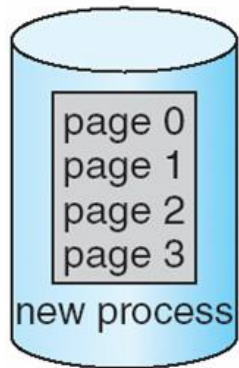
Paging Hardware



Free Frames

free-frame list

14
13
18
20
15

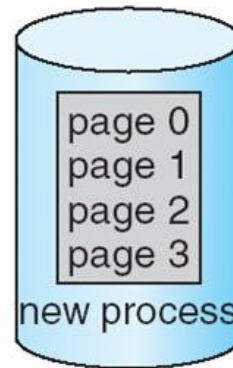


(a)

Before allocation

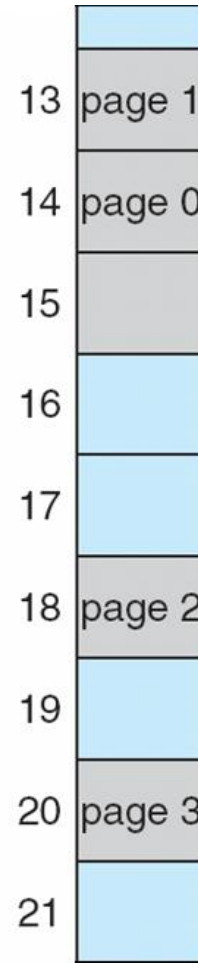
free-frame list

15



0	14
1	13
2	18
3	20

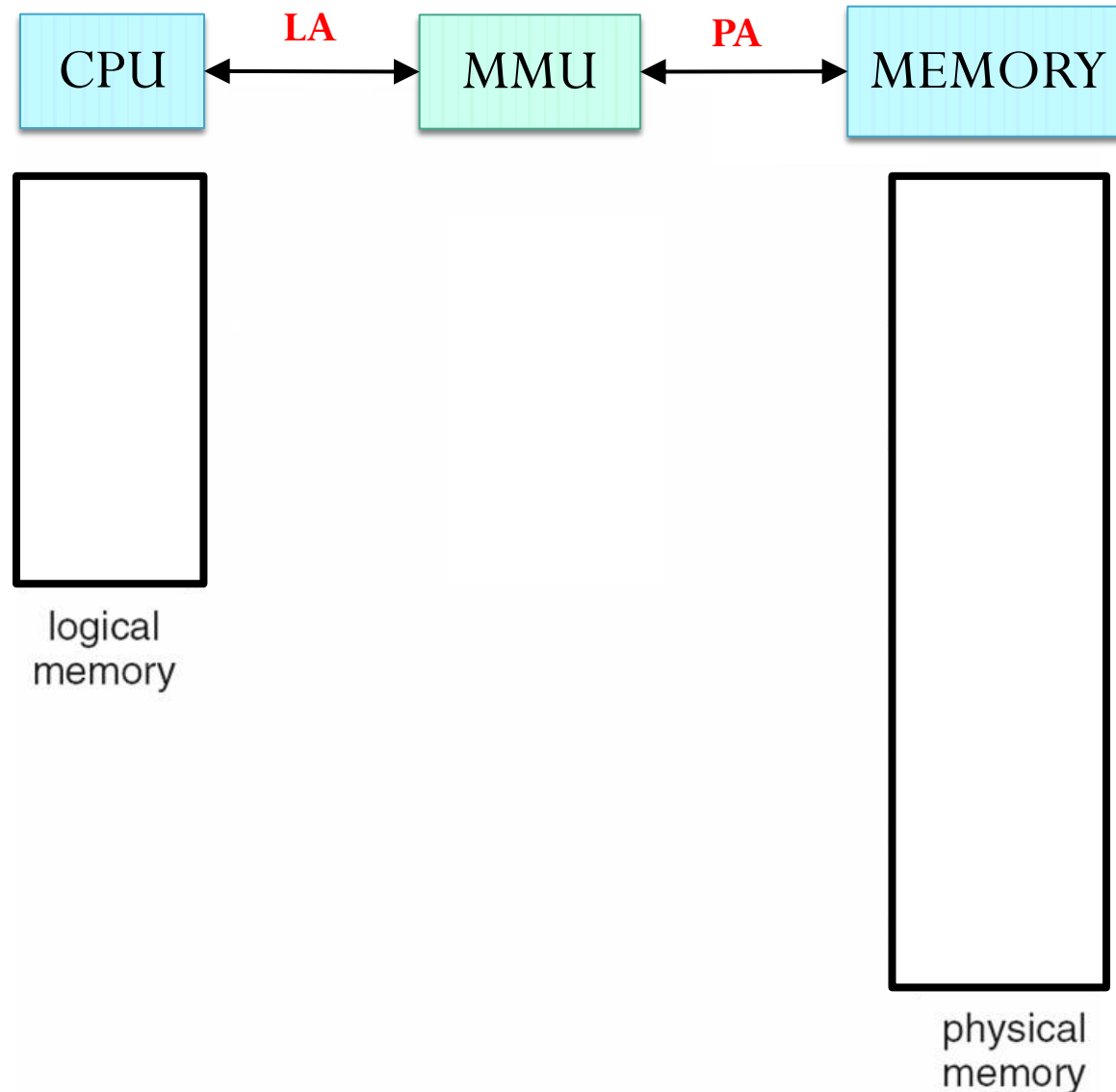
new-process page table



(b)

After allocation

Paging Model of Logical and Physical Memory



Page Table

- Each OS has its own methods for storing page tables
 - Most allocate a page table for each process
 - Each process has a pointer to the page table in PCB

Implementation of Page Table

- Page table is kept in main memory
 - **Page-table base register (PTBR)** points to the page table
 - **Page-table length register (PRLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses. One for the page table and one for the data/instruction.
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**

TLB - Associative Memory

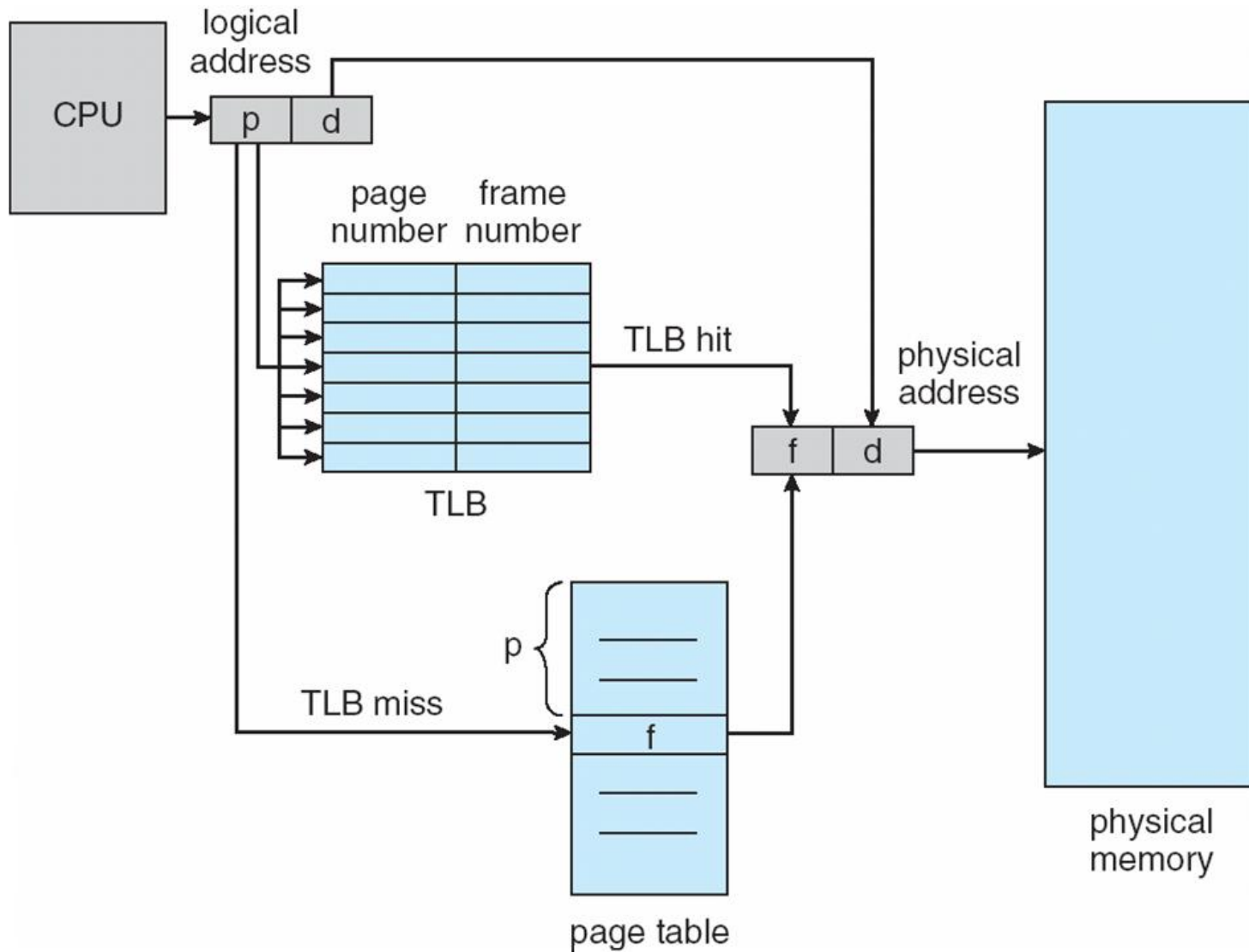
- Associative memory – parallel search

Page #	Frame #

Address translation (p, d)

- If p is in associative register, get frame # out
- Otherwise get frame # from page table in memory

Paging Hardware With TLB

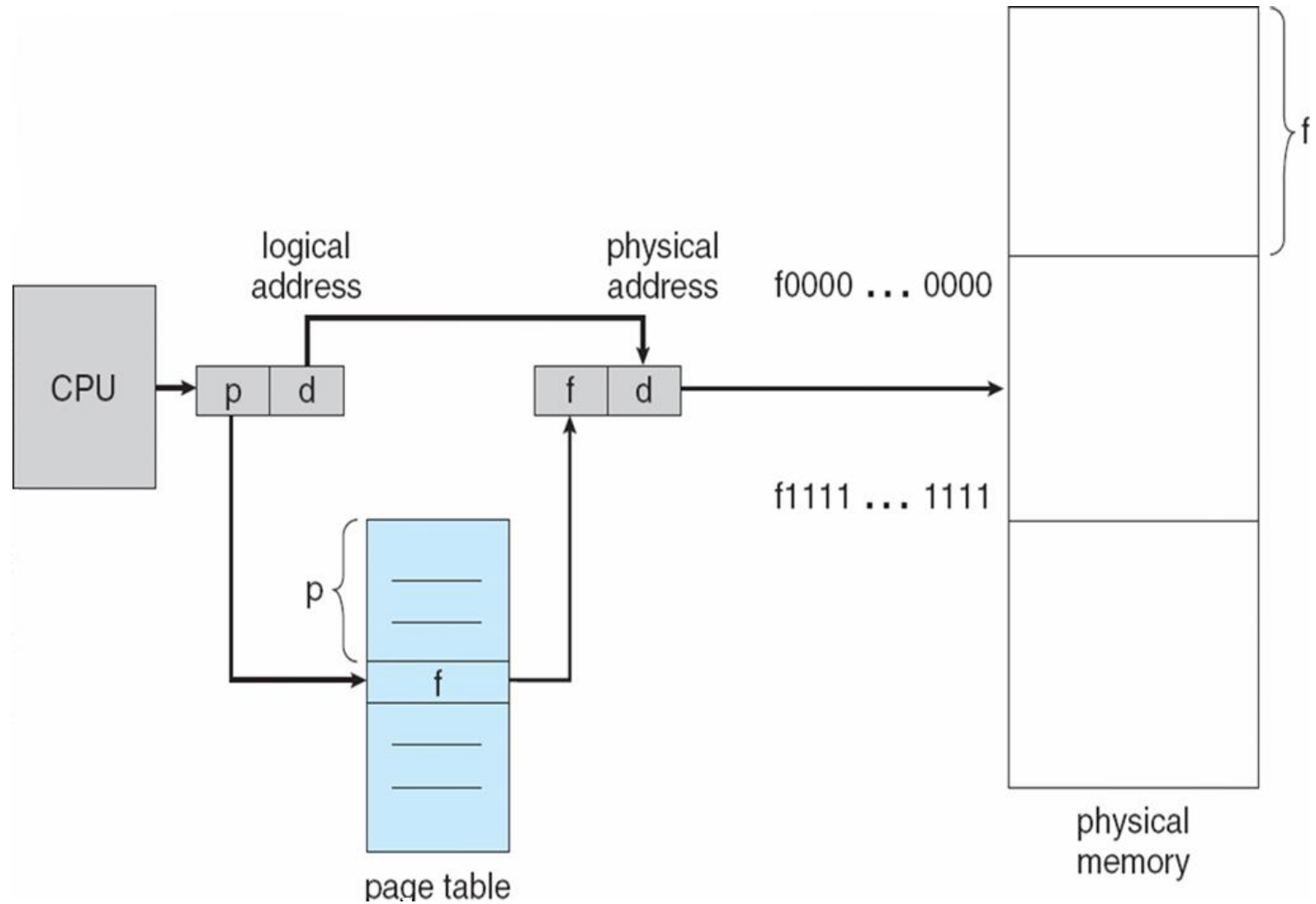


Effective Access Time

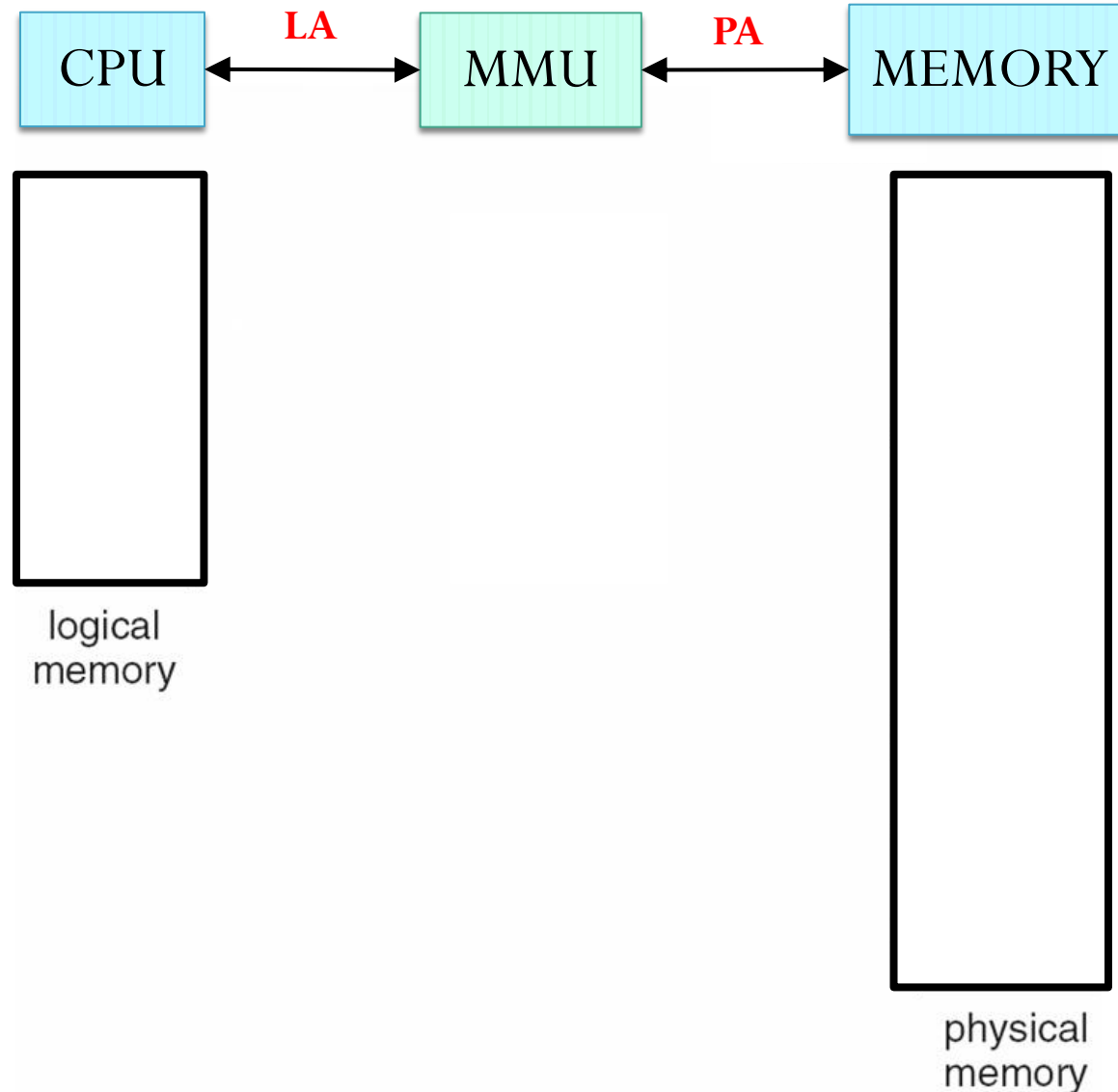
- Preliminaries
 - TLB Lookup = ε time unit
 - Assume memory cycle time is 1 microsecond
 - Hit ratio – percentage of times that a page number is found in the TLB; ratio related to the number of TLB
 - Hit ratio = α
- **Effective Access Time (EAT)**

$$\begin{aligned} \text{EAT} &= (1 + \varepsilon) \alpha + (2 + \varepsilon)(1 - \alpha) \\ &= 2 + \varepsilon - \alpha \end{aligned}$$

Paging Hardware



Paging Model of Logical and Physical Memory



Issues on Page Tables

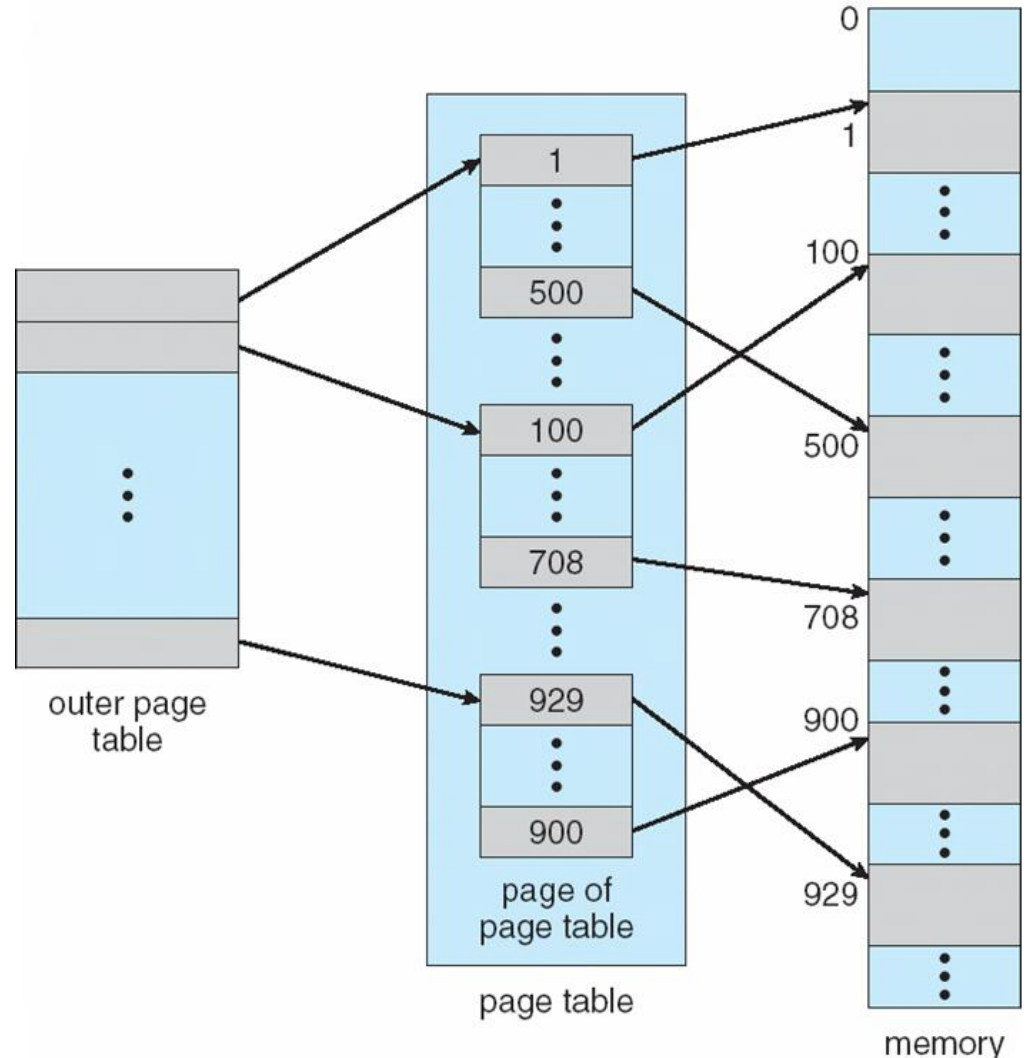
- How big is it? (4B per PT entry)
 - 32-bit address space (4KB per page)
 - 48-bit address space (4KB per page)

Structure of the Page Table

- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table



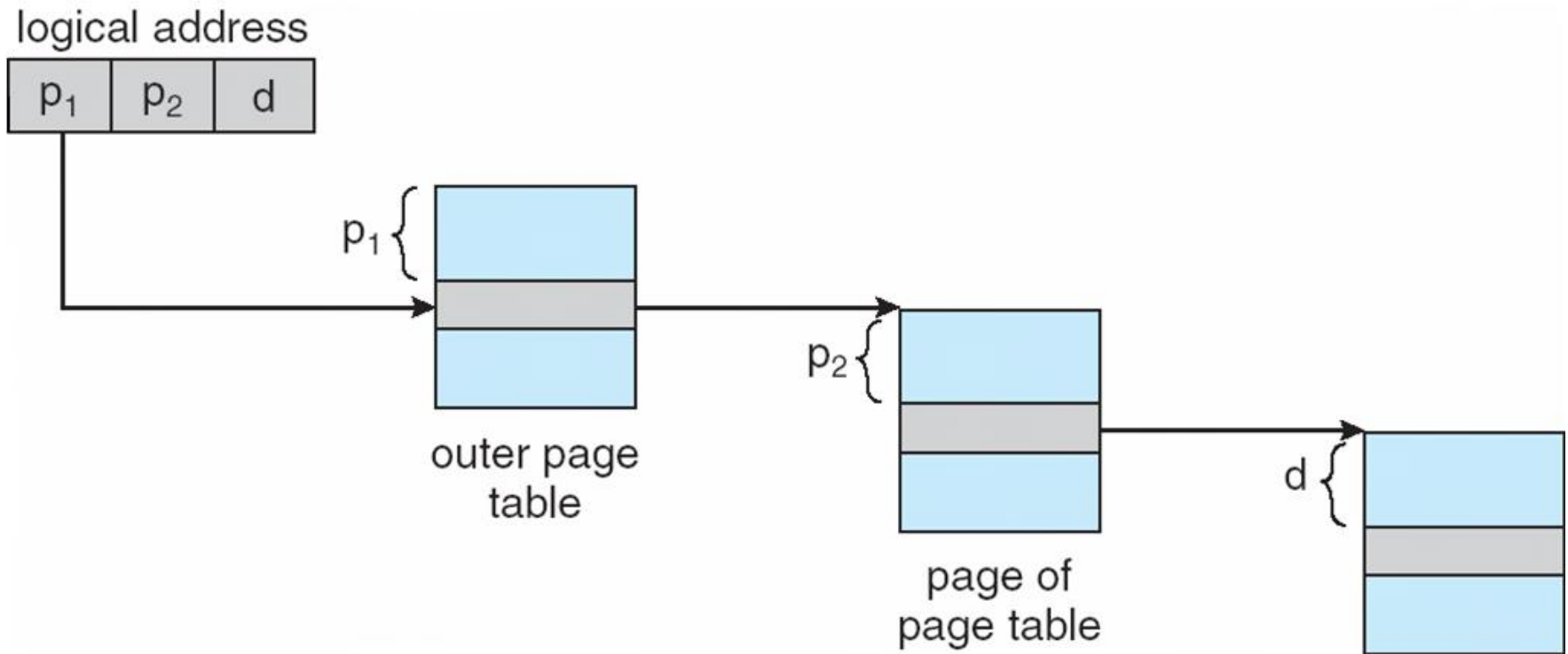
Two-Level Paging Example

- A logical address (on 32-bit machine with 4K page size) is divided into:
 - a page number consisting of 20 bits
 - a page offset consisting of 12 bits
- Since the page table is paged, the page number is further divided into:
 - a 10-bit page number
 - a 12-bit page offset
- Thus, a logical address is as follows:

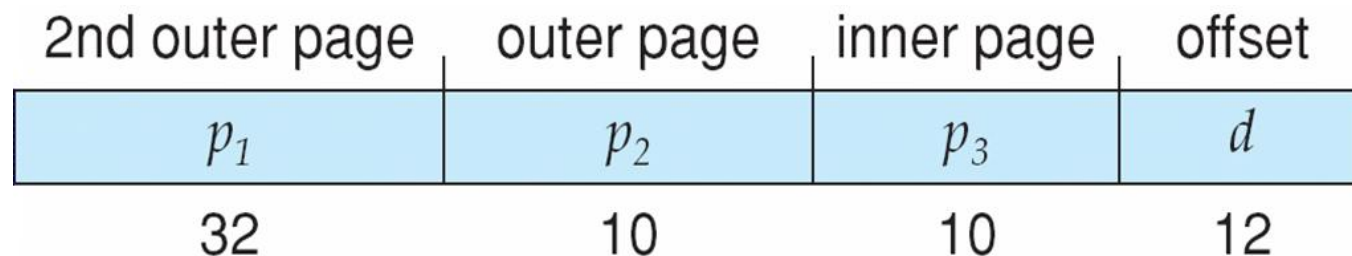
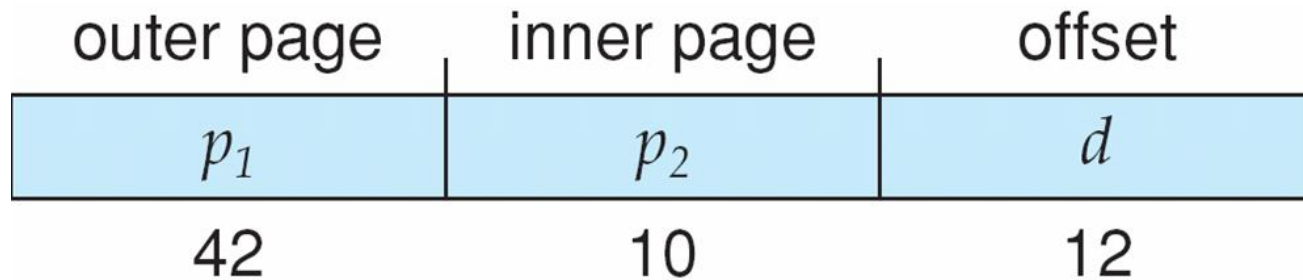
page number		page offset
p_1	p_2	d
10	10	12

where p_1 is an index into the outer page table, and p_2 is the offset within the page of the outer page table

Address-Translation Scheme



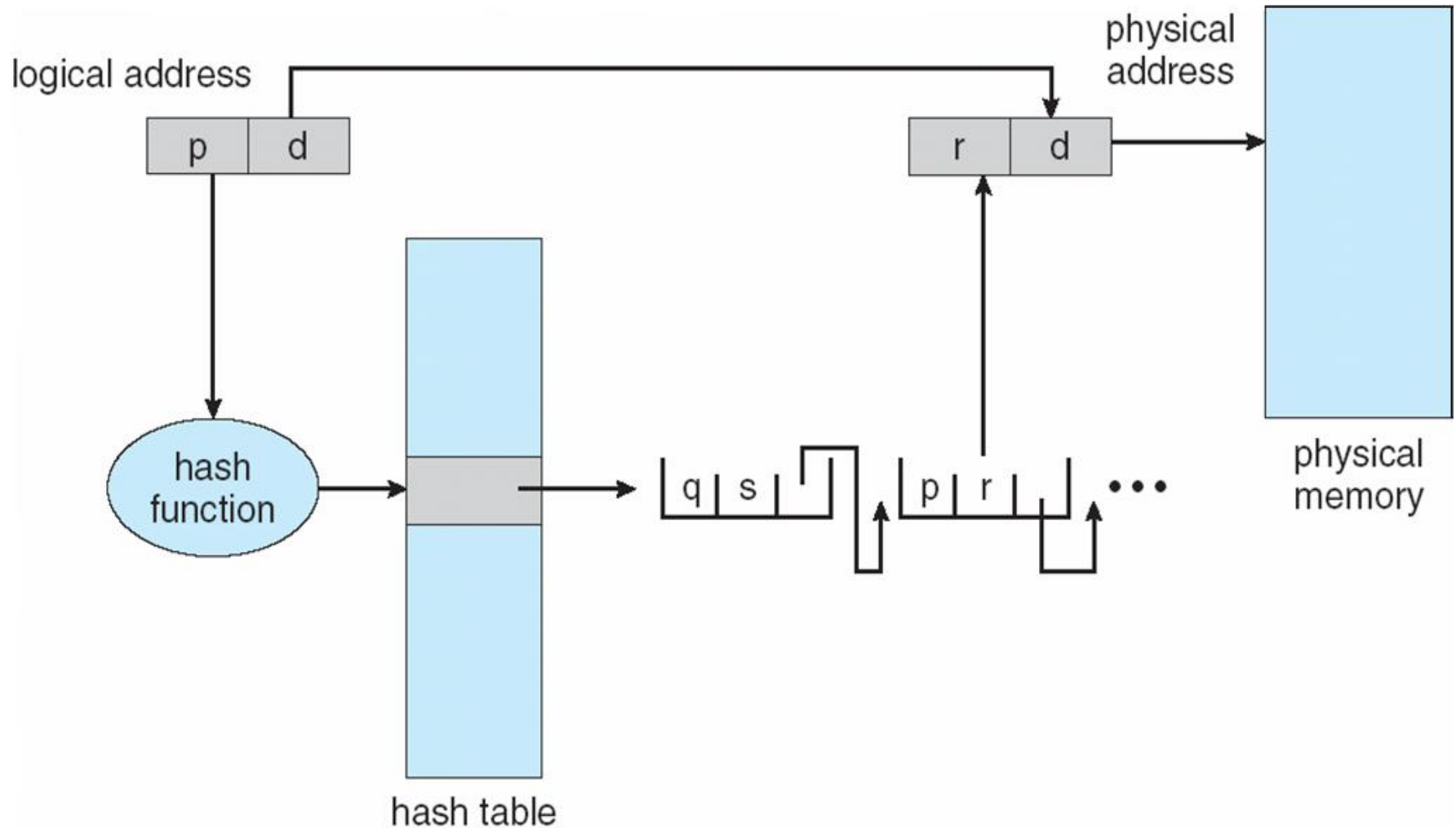
Three-level Paging Scheme



Hashed Page Tables

- Common in address spaces > 32 bits
- The virtual page number is hashed into a page table
 - This page table contains a chain of elements hashing to the same location
- Virtual page numbers are compared in this chain searching for a match
 - If a match is found, the corresponding physical frame is extracted

Hashed Page Table



Inverted Page Table

- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- **Decreases memory** needed to store each page table, but **increases time** needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries

Inverted Page Table Architecture

